Contracts that make composition reliable

Proposed architecture contracts. Current implementation evidence is in CURRENT_STATE; the executable illustrative subset is in `prototype/model.js`. No manifest from this run is installed in the application.

Manifest contract

Every module declaration must identify its stable ID, schema version, public contract version, owner, EN/FR name/outcome, provided capabilities, required and optional dependencies, exclusive capabilities, owned records, interfaces, permissions, scopes, controls, UI contributions, setup/migration steps, country/provider prerequisites, jobs/health, lifecycle, retention/export and commercial references.

The sample registry has all 20 module entries and an intentionally conflicting external-ledger fixture. Versions are pinned to `1.0.0` for demonstration. Its version validator accepts only three nonnegative integer components and explicit inclusive-minimum/exclusive-maximum ranges; unsupported prerelease/build/range syntax fails. It is **not a full SemVer implementation**. Production should adopt one validated range parser consistent with the lockfile and check incompatible contract changes deliberately. Semantic Versioning specifies major versions for incompatible public API changes; module/data migrations still need their own compatibility evidence. Semantic Versioning 2.0.0

Reconciliation example, with production additions

The JSON file contains a runnable compact manifest. This expanded design shows the meaningful operation contract that production must add; names shown below are proposed logical interfaces, not existing endpoints:

```
{
  "schemaVersion": 1,
  "id": "payment_reconciliation",
  "version": "1.0.0",
  "owner": "Finance",
  "name": {"en": "Reconciliation", "fr": "Rapprochement"},
  "outcomes": ["Substantiate recorded cash against independent statements"],
  "requires": [
    {"moduleId": "finance", "min": "1.0.0", "maxExclusive": "2.0.0"},
    {"moduleId": "accounting", "min": "1.0.0", "maxExclusive": "2.0.0"}
  ],
  "provides": ["reconciliation.match", "reconciliation.exceptions.read"],
  "ownedData": ["ReconciliationRun", "ReconciliationMatch"],
  "writesThrough": ["finance payment-evidence API", "ledger posting API"],
  "commands": [{
    "id": "reconciliation.confirmMatch", "version": 1,
    "scope": ["organization", "paymentAccount", "matchCandidate"],
    "controls": ["known permission", "current source revisions", "balanced difference resolution",
"required reviewer separation"],
    "idempotency": ["organizationId", "commandId", "clientRequestId"],
    "requires": ["finance.paymentEvidence.read", "ledger.sourcePosting.read"],
    "effects": "Stores the reviewed association; posts only an explicitly approved adjustment
through the ledger owner"
  }],
  "queries": [{"id": "reconciliation.readExceptions", "version": 1, "sourcePermissionsRequired":
true}],
  "events": [{"id": "reconciliation.matchConfirmed", "version": 1, "partition":
"organizationId:reconciliationRunId"}],
  "surfacePolicy": "Retain existing routes through a registered adapter; use Finance navigation
slot",
  "setup": ["scoped account", "verified import or provider source", "matching policy"],
  "health": ["unprocessed statements", "stale projection age", "failed evidence retrieval"],
  "deactivation": "Stop new matching; retain reviewed associations and source evidence; drain
approved adjustments",
  "commercial": {"packageRef": "to-be-approved", "authorizationSource": false}
}
```

This expanded profile is a design specification, not a second machine-readable schema. Implementers must reconcile it into one production schema before shipping. Example owned names are conceptual; map each to actual Prisma records and services explicitly. The demo's generic lifecycle and job fields are abbreviated; they do not claim full domain behavior is implemented.

Compatibility rules

Rule	Failure reported before activation	Practical resolution
Unique stable IDs and supported schema	Duplicate or malformed manifest.	Registry owner corrects the declaration; do not pick the last definition.
Required dependency closure	Reconciliation without finance/accounting.	Include the required capabilities in the reviewed plan or choose a different outcome.
Acyclic dependency graph	A requires B requires A.	Move common interfaces to the real owner/foundation or replace optional coupling with a query/event.
Compatible pinned versions	A dependency outside the declared range.	Upgrade the compatible set together or retain the old deployment; no silent coercion.
Single authoritative writer	Two modules claim the same authoritative record.	Resolve ownership; use a projection or owner command.
Exclusive capability	Two ledger authorities claim `ledger.authority`.	Keep native posting and use external export/mirroring, or design a separate reviewed external-ledger product profile.
Unique route/contribution ownership	Duplicate route or conflicting navigation contribution.	Reuse one route owner and add contextual links; never last-registration wins.
Commercial eligibility	Selected capability has no applicable eligible grant.	Prepare a quote or select an eligible alternative; no purchase during planning.
Operation prerequisites	Country/provider/data setup absent or invalid.	Name the missing item, responsible person, source and next action.
Removal impact	Required dependent or existing obligation becomes invalid.	Retain dependency, replace with a proven compatible capability or drain/revise the dependent workflow.

The model checks IDs, basic schema, missing dependencies, cycles, range mismatch, ownership/route collisions, exclusivity, selected licensing and sample setup. It does not perform arbitrary JSON schema validation, database verification, signed package verification, policy proofs or all production route-prefix overlaps. Production adds normalized route-pattern overlap checks (including dynamic segments), capability-level prerequisites and exhaustive operation registration.

Valid and rejected examples

Valid: sales + finance + accounting + reports resolves platform and ledger integrity. It serves service invoicing without inventory or POS. Reports require the user's source permissions.

Rejected: sales + a proposed external-ledger authority also brings native ledger integrity through sales. Both claim `ledger.authority`. The affected outcome is one authoritative posting per sale. The prototype blocks review and offers “Keep the built-in ledger.” An external reporting/export adapter could coexist; a second posting authority cannot be introduced by renaming it. The fixture is not an implemented integration.

Commands, queries and events

Interface	Minimum envelope	Ownership and safety
Command	command/version, client request ID, expected object revision, typed business payload. Trusted context supplies actor, organization, permission result and configuration revision.	Caller cannot supply authoritative tenant/approval/grant booleans. Owner validates object scope and invariants, records the result, and reuses the same result on duplicate requests. A reused key with a different payload is rejected.
Query	query/version, scope filters, cursor, authorized field selection, requested freshness.	Owner returns data plus `asOf`, source version and `ready/empty/partial/unavailable/error`; no success-shaped empty list on dependency failure. Cache keys include tenant, actor scope, permissions and relevant revisions.
Event	event ID/type/schema version, organization, aggregate ID/version, occurredAt, correlation/causation IDs, source revision, minimal typed payload.	Record atomically with the committed business change. Do not put credentials, whole employee records or approval assertions from untrusted text in events.
Provider callback	provider connection ID, verified signature/authentication, external event ID, provider object revision and payload digest.	Resolve organization from the trusted connection; record/deduplicate in an inbox. An external event changes evidence/subscription state only through the owner resolver.

The accounting command checks balanced amounts, allowed currency, source uniqueness and period policy before posting. Reconciliation confirms evidence against an existing payment and posts only a distinct approved adjustment. An accounting workspace command delegates to ledger ownership; it cannot update journals directly. Reports and analytics rebuild projections from domain facts and never write those source facts.

Transaction and delivery model

For an operation whose local financial invariants must be atomic, domain owners accept one shared transaction context and complete the authoritative writes and business-event outbox entry together. Keep that boundary narrow and explicit. Read models, notifications, providers and large external work run after commit.

A transactional outbox closes the gap between committing business data and recording its event. Delivery can still duplicate; consumers need deduplication and idempotent effects. This design assumes at-least-once delivery, not universal exactly-once behavior. AWS transactional outbox pattern

Consumers deduplicate `(organizationId, consumerId, eventId)` and use an aggregate version to reject stale effects or hold a gap for reconciliation. Ordering is per declared aggregate, not global. A late payment settlement must not be discarded merely because an unrelated account advanced. Retry transient errors with capped attempts/backoff and jitter; preserve permanent failures in an exception queue with a responsible owner. Replay uses the original operation/event identity, compatible schema upcasting and a recorded replay cause.

External payments use a stable provider idempotency key when supported. After an uncertain timeout, reconcile provider state before creating another payment. Cancellation of a local request does not establish that a provider did nothing. Compensation is an explicit business reversal/refund/adjustment with its own audit; it is not deletion of the original record.

Proposed persistence model

These are conceptual additions, not existing schema declarations or migrations from this run.

Record	Key contents and constraints
`PackageVersion`	Immutable offer version, included capability references, limits, commercial evidence reference. Editing creates a new version.
`SubscriptionSnapshot`	Organization + provider/customer/subscription reference, provider revision, effective interval, source digest/status. Ambiguous or out-of-order events trigger reconciliation.
`ModuleGrant`	Organization, capability/module ID, source reference, status, interval, limits and revocation. Canonical resolution merges valid sources deterministically; no duplicate last-write wins.
`TenantConfigurationRevision`	Organization + monotonically increasing revision, parent/expected revision, selected modules, pinned manifest set, impact digest, creator and time. Immutable after submission.
`ModuleProvisioningStep`	Organization + configuration revision + module + step unique key; attempt, lease, input digest, result/reference and failure classification.
`ModuleLifecycleEvent`	Organization, revision, transition, actor/service identity, reason, scope, evidence reference and correlation. Append-only.
`ModuleEnforcementPolicy`	Versioned tenant/cohort + surface/operation scope, observe/enforce behavior and evidence/authorization reference where required. No implied global switch.
`ProviderInboxReceipt`	Trusted connection, external event ID/revision, digest, receive/verify/reconcile outcome. Reject key reuse with different payload.

Use tenant-inclusive unique keys and foreign keys where the actual schema permits them. All queries and writes still require tenant scope. Maintain an active revision pointer using compare-and-swap; configuration history and in-progress desired revisions are separate. Record approval links to the native register only for changes whose actual policy requires approval. Do not implement another signature store inside module tables.

Large business records stay in existing domain tables. Tenant activation must not run schema migrations. Schema is deployed through the existing migration process, independently verified; per-tenant provisioning configures compatible, already-deployed structures. Expand/backfill/verify/contract sequencing preserves old readers until rollout/rollback requirements are satisfied.

UI, infrastructure and operating contract

A surface declaration includes a unique ID, existing route adapter, canonical navigation slot/order, EN/FR text key, required operation and permission mapping, supported object scope, and standard state behavior. Use the existing permission registry: new illustrative names in the example manifests are not automatically recognized. Privileged code cannot treat an unknown permission as allowed.

Jobs declare bounded batch size/time, cancellation points, retry budget, queue ownership and identity scope. Health exposes pending age, failed attempts, dead-letter count, source freshness and last successful reconciliation without confidential payloads. Set numeric SLOs and limits after measuring the chosen pilot's workload; no invented production latency guarantee appears here.

The prototype is an executable design aid. Its caller-supplied booleans and in-memory revision state are deliberately synthetic. Production must resolve those facts on the server, preserve revocation, and test the actual persistence/concurrency boundary.